



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Normalization

17- Mar -2026



Contents

- Features of Good Relational Designs
- Functional Dependencies
- Non-loss Decomposition
- Normalization
- Denormalization in databases

Features of Good Relational Designs



Features of Good Relational Designs

Relational Designs

- **Relation schemas** are **directly** generated from the **E-R design**.
- **Goodness (or badness)** of the resulting set of schemas depends on **how good was the E-R design**.

Design Alternative: Larger Schemas

- Eg. In University database, there are department and faculty schemas given below:

Department(DepartmentID, Name, Building)

Professor(ProfessorID, Name, Email, DepartmentID)

- Instead of having separate schemas, we can have **Professor_Dept** (ProfessorID, Name, Email, Dept_Name, Building) - natural join on the relations corresponding to Professor and Department.
- Looks like a good idea because some queries can be expressed using **fewer joins** but we have to **repeat** the **department information** for **each** professor/instructor in the department.

Features of Good Relational Designs

Relational Designs

- So, using **Professor_Dept** is a **bad idea** since it stores the **data redundantly** and **create inconsistency**.
- If we are **creating a new department**, we cannot insert department details (Dept_Name, Building) unless that department has at least one professor/instructor at the university.

Design Alternative: Smaller Schemas

- **Not all decompositions are good.**
- Eg. Suppose we decompose Student(ID, name, street, city, email, DOB) into
Student1 (ID, name)
Student2 (name, street, city, email, DOB)
- The **problem** arises when we have **two students** with the **same name**.

Features of Good Relational Designs

Informal Design Guidelines for Relation Schemas

Four informal guidelines to determine the quality of relation schema design:

1. Clear Attribute Semantics

- Make sure that the semantics of the attributes is clear in the schema
- **Poor Design:** Student (id, field1, field2, field3)
- **Improved Design:** Student (ID, Name, Email, DOB)

2. Reduce Redundant Information

- Avoid repeating data across tuples
- **Example:** Enrollment (EnrollmentID, StudentID, CourseID, Semester, Grade)
- Stores only foreign keys StudentID, CourseID instead of duplicate data of student and course details.
- Updates to student/course details propagate automatically.

Features of Good Relational Designs

Informal Design Guidelines for Relation Schemas

3. Minimize NULL Values

- Design schemas to **avoid optional attributes**.
- **Example:** Many Professors won't have research or funded projects, so have Professor (ProfessorID, Name, Email, DepartmentID) and Projects (ProfessorID, Fund_Amount)

4. Prevent Spurious Tuples

- **Disallowing** the possibility of generating **spurious tuples** (additional tuples that are produced when two tables are joined)
- **Example:** Teaches (ProfessorID, CourseID, DepartmentID) and Course (CourseID, Title), when joining Teaches and Course, we get fake combinations because DepartmentID in Teaches isn't constrained to match the course's true department.

Functional Dependencies



Functional Dependencies

Dependency

- A dependency in the context of a database management system (DBMS) refers to a **relationship between attributes or sets of attributes** within a **database table**.
- Dependencies help establish how **data elements relate to each other**.
- There are different types of dependencies:
 - ✓ Functional dependency
 - ✓ Partial dependency
 - ✓ Transitive Dependency
 - ✓ Multi-valued Dependency

Functional Dependencies

Functional dependency

- ✓ **Functional Dependency** occurs when the **value of one attribute uniquely determines the value of another attribute** in a database table. In other words, if **two tuples (rows) have the same value for one attribute**, they must have the **same value for another attribute**.

- **Example:**

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Dr.Smith
2	Bob	Math 101	Dr.Smith
3	Charlie	Science 101	Dr.Jones
4	David	English 101	Dr.Johnson
5	Emily	History 101	Dr.Wilson

Functional Dependencies

Functional dependency

Student_ID $\rightarrow$ Student_Name

Course_Name $\rightarrow$ Instructor_Name

- This means that given a **Student_ID**, you can **uniquely determine** the **Student_Name**, and given a **Course_Name**, you can **uniquely determine** the **Instructor_Name**.

Functional Dependencies

Partial dependency

- ✓ **Partial Dependency** occur when **non-key attributes** depend on **part of a composite primary key**, leading to **redundancy** and **update anomalies**.

- **Example:**

Student_ID	Student_Name	Course_Name	Instructor Name	Grade
1	Alice	Math 101	Dr.Smith	A
2	Bob	Math 101	Dr.Smith	B
3	Charlie	Science 101	Dr.Jones	A
4	David	English 101	Dr.Johnson	B
5	Emily	History 101	Dr.Wilson	A

Functional Dependencies

Partial dependency

- **Composite Primary Key:** let's define the **composite primary key** as (Student_ID, Course_Name).
- **Attributes:**
 - **Student_Name:** Dependent only on Student_ID (not the full composite key).
 - **Instructor_Name:** Dependent only on Course_Name (not the full composite key).
 - **Grade:** Dependent on the full composite key (Student_ID, Course_Name).
- **Partial Dependency:**
 - ✓ Student_Name is partially dependent on Student_ID:

For example, Student_ID = 1 determines Student_Name = Alice. The Student_Name does not depend on the Course_Name part of the composite key.

Functional Dependencies

Partial dependency

✓ Instructor_Name is partially dependent on Course_Name:

For example, Course_Name = Math 101 determines Instructor_Name = Dr. Smith. The Instructor_Name does not depend on the Student_ID part of the composite key.

- In this example, both **Student_Name** and **Instructor_Name** exhibit **partial dependency** on parts of **the composite primary key**.

Functional Dependencies

Transitive Dependency

✓ **Transitive Dependency** occurs when an **attribute** is **functionally dependent** on another attribute that is not part of the **primary key**.

- **Example:**

Student_ID	Student_Name	Course_Name	Instructor Name	Grade
1	Alice	Math 101	Dr.Smith	A
2	Bob	Math 101	Dr.Smith	B
3	Charlie	Science 101	Dr.Jones	A
4	David	English 101	Dr.Johnson	B
5	Emily	History 101	Dr.Wilson	A

Functional Dependencies

Transitive Dependency

$\text{Student_ID} \rightarrow \text{Student_Name} \rightarrow \text{Course_Name}$

$\text{Student_ID} \rightarrow \text{Student_Name} \rightarrow \text{Instructor_Name}$

- In both cases, **Course_Name** and **Instructor_Name** are **transitively dependent** on **Student_ID** through **Student_Name**.

Functional Dependencies

Multi-Valued Dependency

- ✓ **Multi-Valued Dependency** exists when an **attribute set functionally determines another attribute set**.
- **Example:**

Student_ID	Student_Name	Course_Name	Instructor Name	Hobby
1	Alice	Math 101	Dr.Smith	Painting
1	Alice	Math 101	Dr.Smith	Chess
2	Bob	Math 101	Dr.Smith	Soccer
2	Bob	Math 101	Dr.Smith	Chess
3	Charlie	Science 101	Dr.Jones	Reading
4	David	English 101	Dr.Johnson	Writing
5	Emily	History 101	Dr.Wilson	Painting
5	Emily	History 101	Dr.Wilson	Soccer

Functional Dependencies

Multi-Valued Dependency

- **Primary Key:** The primary key could be (Student_ID, Course_Name, Hobby).
- **Attributes:**
 - Student_ID: Unique identifier for each student.
 - Student_Name: Name of the student.
 - Course_Name: Name of the course the student is enrolled in.
 - Instructor_Name: Name of the instructor for the course.
 - Hobby: A hobby that the student has.

Functional Dependencies

Multi-Valued Dependency

- **Multi-valued Dependency:**
 - ✓ **Student_ID $\twoheadrightarrow$ Hobby (Student_ID multi-determines Hobby):** For a given **Student_ID**, there are **multiple values of Hobby**, and these values are independent of the **Course_Name** and **Instructor_Name**.
 - ✓ For example, **Student_ID = 1** has **hobbies Painting and Chess**, independently of their course enrollment in Math 101.
 - ✓ **Similarly, Student_ID $\twoheadrightarrow$ Course_Name:** For a given **Student_ID**, there are **multiple values of Course_Name**, and these values are independent of the **Hobby**.
 - ✓ For example, if we expanded the table to show Alice enrolled in multiple courses, these courses would be listed independently of her hobbies.

Non-loss Decomposition



Non-loss Decomposition

What is a Non-loss Decomposition ?

- **Decomposition** - division of **relations into multiple relation** to produce **consistency** in the data.
- **Properties** of Decomposition
 - ✓ Lossless
 - ✓ Dependency Preservation
 - ✓ Lack of Data Redundancy

Lossless or Non-loss Decomposition

- Decomposition of a relation R into relations R1 and R2 such that natural join of relation R1 and R2 will **return the original relation R**.
- **Effective in removing redundancy** from databases while preserving the original data.
- Able to **reconstruct** the relation R from decomposed tables R1 and R2 by using **joins**.

Non-loss Decomposition

Non-loss Decomposition : Example

- Consider a relation Student:

Student_ID	Name	Course
1	Alice	Physics
2	Bob	Chemistry

- Decompose Student into: StudentInfo(StudentID, Name) and Enrollment(StudentID, Course)
- Decomposed tables: StudentInfo and Enrollment**

Student_ID	Name
1	Alice
2	Bob

Student_ID	Course
1	Physics
2	Chemistry

- When we join on StudentID, we get the exact original Student table back - lossless decomposition.**

Non-loss Decomposition

What is a Lossy Decomposition ?

Lossy decomposition

- Join operation on the sub-relations doesn't result to the same relation which was decomposed.
- After the join operation, some extraneous (spurious) tuples are found.
- Difficult for the user to identify the original tuples because of extra tuples.

Lossy decomposition – Example

- Consider a relation CourseAssignments:

Student_ID	Course	Instructor
1	Physics	Dr. Smith
2	Chemistry	Dr. Lee
3	Physics	Dr.Winston

- Decompose into: StudentCourse(StudentID, Course) and CourseInstructor(Course, Instructor)

Non-loss Decomposition

Lossy Decomposition : Example

- Decomposed tables: StudentCourse and CourseInstructor

Student_ID	Course
1	Physics
2	Chemistry
3	Physics

Course	Instructor
Physics	Dr. Smith
Chemistry	Dr. Lee
Physics	Dr.Winston

- If we join on Course, the join might create spurious tuples - lossy decomposition

Student_ID	Course	Instructor
1	Physics	Dr. Smith
1	Physics	Dr.Winston
2	Chemistry	Dr. Lee
3	Physics	Dr. Smith
3	Physics	Dr.Winston

Non-loss Decomposition

Quiz



1) A good relational database design should aim to:

a) Maximize data redundancy

b) Minimize data redundancy and prevent anomalies

c) Store data in multiple unrelated tables

d) Eliminate the need for keys

Answer : Option b)

Non-loss Decomposition

Quiz



2) In a good relational design, Functional Dependency ensures that:

a) All attributes are independent of each other

b) One attribute uniquely determines another attribute

c) No attribute depends on the primary key

d) Attributes are multi-valued

Answer : Option b)

Non-loss Decomposition

Quiz



3) Which of the following is an example of Functional Dependency?

a) $\text{StudentID} \rightarrow \text{StudentName}$

b) $\text{Age} \rightarrow \text{StudentID}$

c) $\text{StudentName} \leftrightarrow \text{CourseCode}$

d) None of the above

Answer : Option a)

Non-loss Decomposition

Quiz



4) Non-loss Decomposition means:

a) Data is duplicated when splitting tables

b) Data is lost during decomposition

c) Decomposing tables does not lose any data and original information can be reconstructed

d) Tables are split without any rules

Answer : Option c)

Non-loss Decomposition

Quiz



5) Why is Non-loss Decomposition important in database design?

a) It prevents data redundancy

b) It allows faster query processing

c) It ensures that no information is lost when a table is split into multiple tables

d) It avoids the use of keys

Answer : Option c)

Normalization



Normalization

What is Normalization?

- **Normalization in database design** is the process of **organizing data to reduce data redundancy and improve data integrity.**



What is Data Redundancy and Data Integrity?

Data redundancy refers to the **unnecessary repetition of data** within a database or storage system. That is the **same piece of data is stored in multiple places.**

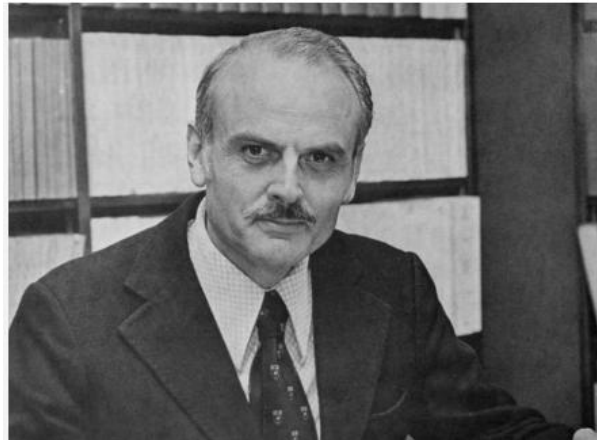
Data integrity means that the **data remains unchanged** and it can be refer to the **accuracy, consistency, and reliability of data**



Normalization

What is Normalization?

- **Reduce in data redundancy and improvement in data integrity** is achieved by **dividing large tables into smaller, more manageable pieces** while establishing **relationships** between them.
- The main **goals of normalization** are to ensure that **data dependencies** are **logical** and that the **database is efficient in storing, retrieving, and maintaining data**.
- The inventor of the **relational model Edgar Codd** proposed the **theory of normalization of data**.



Normalization

What is Normalization?

- The process of normalization involves **several stages**, known as **normal forms**, each with specific requirements and benefits.
- **Common Normal Forms :**
 - **First Normal Form 1NF**
 - **Second Normal form 2NF.**
 - **Third Normal Form 3NF.**
- Later, **Edgar Codd** joined **Raymond F. Boyce** to **develop** the **theory of Boyce-Codd Normal Form**.
- After that we have **Fourth, Fifth and Sixth normal forms**.

Normalization

Why normalization is important in SQL?

- If a table is **not properly normalized** and has **data redundancy**, it will **not only take up extra data storage space** but also make it **difficult to handle** and **update the database**.
- **Example:** In a university database, if all student enrollments are stored in a single table like **StudentEnrollments** where each course registration duplicates the student's personal details (e.g., name, email, major). For example, if "Anitha" enrolls in five courses, her name, email, and major would be **repeated** five times. When Anitha updates her email, every duplicate record must be modified, risking **inconsistencies** if some entries are missed. This redundancy also **wastes storage** and slows down queries, as the database must scan redundant data to retrieve simple information (e.g., listing all courses for one student).
- This leads to **data redundancy, increased storage requirements, data inconsistency, and inefficient data retrieval** when accessing the **`StudentEnrollments`** table.

Normalization

Why normalization is important in SQL?

- Similarly, there are **several factors** as follows that drive the **need for normalization**:
 1. **Insertion, Deletion, and Update Anomalies**
 2. **Difficulty in managing the relationship**
 3. **Dependency**

Normalization

Why normalization is important in SQL?

What is Anomalies?



Anomalies in the **relational model** of databases are **inconsistencies** or **errors** that happen because of **poor database design**. These issues come up when **data isn't organized correctly** into related tables but is **instead stored in a flat structure**.



Normalization

Why normalization is important in SQL?

What kind of problems anomalies can cause?

Well, Anomalies can create the **insertion anomaly** that is it can **lead to redundant data** being stored unnecessarily, another common problem **update anomalies** is difficulties in **updating information accurately**, Finally with deletion anomaly **Improperly structured databases** can also result in **unintended data loss** when deleting information.

You can see in detail with this types of anomalies in following slides



Normalization

Why normalization is important in SQL?

- **Insertion Anomaly:** An **insertion anomaly** is a type of **data inconsistency** that can occur in a database when certain **conditions** make it **difficult to insert data without introducing redundancy** or **leaving certain required fields blank**. This **anomaly arises** primarily in databases that **are not properly normalized**.
- **Example:** Consider a table storing information about students and the courses they are taking. The table might look like this:

Student_ID	Student_Name	Course_ID	Course_Name
1	John Doe	101	Math 101
2	Jane smith	102	History 101

Normalization

Why normalization is important in SQL?

- ✓ **Scenario Leading to Insertion Anomaly:** Suppose you want to add a **new course to the database**, but there are **no students enrolled** in this course yet. **Without normalization**, you might be **forced to insert a row with null values for the student-related columns**, or **duplicate information**, which **leads to an insertion anomaly**.

Student_ID	Student_Name	Course_ID	Course_Name
1	John Doe	101	Math 101
2	Jane smith	102	History 101
Null	Null	103	Science 101

- This is an **insertion anomaly** because:
 - ✓ It compromises **data integrity** by introducing **incomplete records** with **null values** and **increases redundancy** by **duplicating information** when new students enroll in courses, leading to **inefficiencies** and **potential inconsistencies** in the database.

Normalization

Why normalization is important in SQL?

- **Deletion Anomaly:** A **deletion anomaly** in a database occurs when the **deletion** of a piece of data **inadvertently leads** to the **loss of additional, unintended data**. This typically happens when a **database is not properly normalized** and **stores redundant data** within a **single table**. The problem arises because **deleting a record** that contains **necessary information also removes** that information from the database entirely, even if it is still needed elsewhere.
- **Example:**
 - ✓ Consider a table storing information about students and the courses they are enrolled:

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Dr.Smith
2	Bob	Math 101	Dr.Smith
3	Charlie	Science 101	Dr.Jones

Normalization

Why normalization is important in SQL?

- ✓ In the above table, **information** about **courses** and **instructors** is **repeated** for **each student enrolled** in the course.
- ✓ **Scenario Leading to a Deletion Anomaly:** If **Charlie** decides to drop "**Science 101**," and we delete his record from the table, we lose not just **Charlie's** enrollment information but also all details about "**Science 101**" and "**Dr. Jones**":

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Dr.Smith
2	Bob	Math 101	Dr.Smith

- This is a **Deletion anomaly** because:
 - ✓ Now, there is **no record** of "**Science 101**" or "**Dr. Jones**" in the **database**, even though that **information might still be needed**.

Normalization

Why normalization is important in SQL?

- **Update Anomaly:** A **update anomaly** in a database occurs when **data is redundantly stored** and **inconsistencies** arise during updates. These anomalies can lead to **data inconsistency** and **integrity issues**, making it difficult to maintain and query the database accurately.
- **Example:**
 - ✓ Consider a table storing information about students and the courses they are enrolled:

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Prof.Smith
2	Bob	Math 101	Prof.Smith
3	Charlie	Science 101	Dr.Jones
4	David	English 101	Dr.Johnson
5	Emily	History 101	Dr.Wilson

Normalization

Why normalization is important in SQL?

- **Scenario Leading to a update Anomaly:** In the above table, Imagine **Prof. Smith** gets a **promotion** and their title changes to **Dr. Smith**. You would need to **update Dr. Smith's title in every row** where they are listed as the instructor (**Math 101** for students **Alice** and **Bob**).
- If you **miss updating even one instance (say, for Bob)**, the **table becomes inconsistent**. It would show **Dr. Smith** for **Alice** but **Prof.Smith** for **Bob**, even though they have the same instructor.

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Dr.Smith
2	Bob	Math 101	Prof.Smith
3	Charlie	Science 101	Dr.Jones
4	David	English 101	Dr.Johnson
5	Emily	History 101	Dr.Wilson

Normalization

Why normalization is important in SQL?

- This is an **update anomaly** because:
 - ✓ **Updating one piece of data** (instructor title) **requires changes in multiple rows** (every row with Dr. Smith).
 - ✓ **Missing an update** in any **one row leads to inconsistent data** within the table regarding the instructor.

Normalization

Why normalization is important in SQL?

Normalization to prevent anomalies:

- **Normalization** is indeed the **key technique** used in **database design** to **prevent anomalies** such as **insertion, update, and deletion anomalies**.
- In the above Examples, having a **separate table for courses and instructors**, ensuring each piece of **data is stored in only one place**, and using **relationships to link** them leads to **eliminating these anomalies** by **ensuring that data is stored** in a **structured way** that supports **efficient and accurate data operations**.

Normalization

Why normalization is important in SQL?

Difficulty in managing the relationship

- It becomes more challenging to **maintain complex relationships** in an **unnormalized structure**. For example, managing **many-to-many relationships** like **students and courses** can lead to **redundancy and complexity**.
- **Example:** A single table combining students and courses might look like:

Student_ID	Student_Name	Course_Name	Instructor_Name
1	Alice	Math 101	Dr.Smith
1	Alice	History 101	Dr.Wilson
2	Emily	Math 101	Dr.Smith

Normalization

Why normalization is important in SQL?

- ✓ **Redundancy:** Repeatedly storing the same course information for each student enrolled leads to data duplication.
- ✓ **Complexity:** Handling updates, deletions, and insertions becomes cumbersome and error-prone, as changes must be replicated across multiple rows.
- So, **normalizing the database** and separating **related data into distinct tables**, you can effectively manage **many-to-many relationships**, **reduce redundancy**, and simplify **database maintenance**.

Normalization

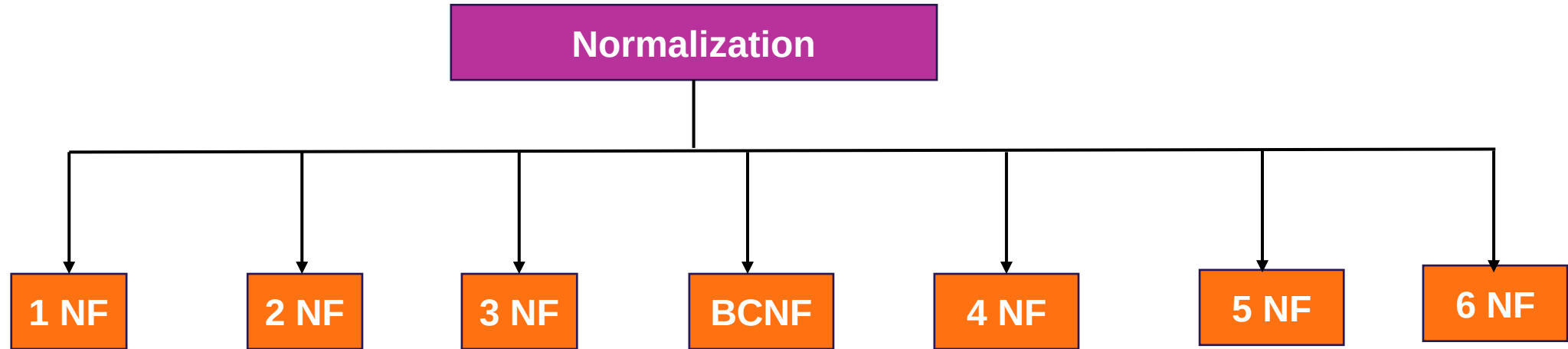
Why normalization is important in SQL?

- Normalization is vital in the realm of database design, and its significance can be summarized as follows.
 - **Reduces redundancy:** Redundancy is when the **same information** is **stored multiple times**, and a good way of avoiding this is by **splitting data into smaller tables**.
 - **Improves query performance:** You can perform faster query execution on smaller tables that have undergone normalization.
 - **Minimizes update anomalies:** With normalized tables, you can easily update data without affecting other records.
 - **Enhances data integrity:** It ensures that data remains consistent and accurate.

Normalization

Types of Normalization

- Normalization is categorized as follows:



First Normal Form (1NF)



Normalization

First Normal Form (1NF)

First Normal Form (1NF)

- 1NF normalization level ensures that **each column in your data contains only atomic values**. **Atomic values** in this context means that **each entry in a column is indivisible**.
- It is like saying that **each cell** in a **spreadsheet** should **hold just one piece of information**.
- 1NF ensures **atomicity of data**, with **each column cell containing only a single value** and each **column having unique names**.
- **Example:** Considering the **University database** to manage academic operations, student enrollments, course offerings, faculty assignments, and department structures, the table has attributes **StudentID**, **StudentName**, **Email**, **DOB**, **Major**, **CourseDetails**, **Grades**, **Professors**, **DepartmentDetails**, **Prerequisites**.

Normalization

First Normal Form (1NF)

- The table for University database is given below:

Student ID	Student Name	Email	DOB	Major	Course Details	Grades	Professors	Department Details	Prerequisites
1001	Arjun	a@u.com	2002-05-01	CS	C101-DBMS-4, C102-OS-3	A, B+	Rao-C101, Sharma-C102	D1-CS-Block A	C102→ C101
1002	Meera	m@u.com	2001-08-11	IT	C101-DBMS-4	B	Rao-C101	D2-IT-Block B	—

In this table:

- CourseDetails contain multiple courses per student
- Grades, Professors, DepartmentDetails, and Prerequisites are all nested or multi-valued

Normalization

First Normal Form (1NF)

- **Solution: Eliminate multivalued and nested fields so that it contains atomic values only**
- CourseDetails attribute can be split into individual attributes as CourseID, CourseTitle, Credits
- DepartmentDetails attribute can be split into individual attributes as DeptID, DeptName, Building
- Grade, ProfessorName, PrerequisiteID are included for each course registered by the student

Student ID	Name	Email	DOB	Major	Course ID	Course Title	Credits	Grade	ProfessorName	DeptID	Dept Name	Building	PrerequisiteID
1001	Arjun	a@u.com	2002-05-01	CS	C101	DBMS	4	A	Rao	D1	CS	Block A	NULL
1001	Arjun	a@u.com	2002-05-01	CS	C102	OS	3	B+	Sharma	D1	CS	Block A	C101
1002	Meera	m@u.com	2001-08-11	IT	C101	DBMS	4	B	Rao	D2	IT	Block B	NULL

Second Normal Form (2NF)



Normalization

Second Normal Form (2NF)

Second Normal Form (2NF)

- This level of normalization builds upon 1NF by **ensuring there are no partial dependencies on the primary key.**
- In simpler terms, **all non-key attributes must depend on the entire primary key** and not just part of it.
- From the **1NF** that was implemented, the **primary key** is a **composite key** (StudentID, CourseID)
- **Partial dependencies:**
 - Attributes like Name, Email, DOB, Major, DeptID, DeptName, Building depend only on StudentID
 - Attributes like CourseTitle, Credits, PrerequisiteID, ProfessorName depend only on CourseID
- **Full dependency:** Attribute Grade depends both on StudentID and CourseID

Normalization

Second Normal Form (2NF)

Solution:

- We need to ensure there are **no partial dependencies on the primary key**.
- This can be done by introducing a **separate table for Student and Course**
- **Student**(StudentID, Name, Email, DOB, Major, DeptID, DeptName, Building)
- **Course**(CourseID, CourseTitle, Credits, PrerequisiteID, ProfessorName)
- We also need to achieve the **many-to-many relationship** between **Student and Course** because each student can enroll in multiple courses and one course can be enrolled by multiple students. This can be done by introducing a separate table **Enrollment(StudentID, CourseID, Grade)**
- Enrollment table establishes a **clear relationship** between **Student and Course**.

Normalization

Second Normal Form (2NF)

- Student Table:**

StudentID	Name	Email	DOB	Major	DeptID	DeptName	Building
1001	Arjun	a@u.com	2002-05-01	CS	D1	CS	Block A
1002	Meera	m@u.com	2001-08-11	IT	D2	IT	Block B

- Course Table:**

CourseID	CourseTitle	Credits	ProfessorName	PrerequisiteID
C101	DBMS	4	Rao	NULL
C102	OS	3	Sharma	C101

- Enrollment Table:**

StudentID	CourseID	Grade
1001	C101	A
1001	C102	B+
1002	C101	B

Third Normal Form (3NF)



Normalization

Third Normal Form (3NF)

Third Normal Form (3NF)

- **3NF builds on 2NF by eliminating transitive dependencies.** A **transitive dependency** occurs when a **non-key attribute** depends on **another non-key attribute**, which in turn depends on the **primary key**.
- From the **2NF we already implemented**, there are **three tables** in our University database.
- The 2NF structure looks efficient, but there is **transitive dependency** in **Student** table.
- **Transitive dependencies:**
 - StudentID determines DeptID and DeptID determines DeptName (StudentID → DeptID → DeptName)
 - StudentID determines DeptID and DeptID determines Building (StudentID → DeptID → Building)

Normalization

Third Normal Form (3NF)

Solution:

- We can split the student table into **two tables: student table and department table**
- Include the **DeptID as foreign key in student table** which refers to primary key DeptID in department table.
- Below is the **updated table**:
- **Student Table:**

StudentID	Name	Email	DOB	Major	DeptID
1001	Arjun	a@u.com	2002-05-01	CS	D1
1002	Meera	m@u.com	2001-08-11	IT	D2

Normalization

Third Normal Form (3NF)

Department Table:

DeptID	DeptName	Building
D1	CS	Block A
D2	IT	Block B

Course Table:

CourseID	CourseTitle	Credits	ProfessorName	PrerequisiteID
C101	DBMS	4	Rao	NULL
C102	OS	3	Sharma	C101

Enrollment Table:

StudentID	CourseID	Grade
1001	C101	A
1001	C102	B+
1002	C101	B

Normalization

Quiz



1) What is the purpose of Normalization in database design?

a) To increase data redundancy

b) To organize data and reduce redundancy and anomalies

c) To store data randomly

d) Eliminate the need for keys

Answer : Option b)

Normalization

Quiz



2) First Normal Form (1NF) requires that:

a) Each column contains atomic (indivisible) values

b) There are no primary keys

c) Tables can have multi-valued attributes

d) Attributes are grouped together

Answer : Option a)

Normalization

Quiz



3) Second Normal Form (2NF) is achieved when:

a) The table is in 1NF and every non-key attribute is fully functionally dependent on the primary key

b) The table is in 1NF and attributes have partial dependency

c) Tables have no primary key

d) Attributes are multi-valued

Answer : Option a)

Normalization

Quiz



4) Third Normal Form (3NF) ensures that:

a) Every non-prime attribute is non-transitively dependent on the primary key

b) There are transitive dependencies

c) Primary key is not needed

d) All attributes are composite

Answer : Option a)

Normalization

Quiz



5) Which of the following is true about normalization?

a) It increases data redundancy

b) It helps remove insertion, update, and deletion anomalies

c) It makes data storage unstructured

d) It removes all keys from the tables

Answer : Option b)

Boyce-Codd Normal Form(BCNF)



Normalization

Boyce-Codd Normal Form(BCNF)

Boyce-Codd Normal Form(BCNF)

- **BCNF** is based on **functional dependencies (FD)** that **consider all candidate keys** in a **relationship**.
- FDs are very important because they guide the **process of normalization** by identifying **dependencies and ensuring data is appropriately distributed across tables**.
- **BCNF** is a **stricter version of 3NF**.
- It ensures that **every determinant** (a set of attributes that uniquely identify a row) in a **table** is a **candidate key** (a minimal set of attributes that uniquely identify a row).
- The whole essence of this is that all **determinants should be able to serve as primary keys**.
- In other words, If $X \longrightarrow Y$ (X determines Y) then **X must be a candidate key** of the relation.

Normalization

Boyce-Codd Normal Form(BCNF)

- While the 3NF structure is good, there might be a **hidden determinant** in the **Course table**.
- In Course table, If **ProfessorName** → **CourseID** (meaning each professor teaches only one course), this would violate BCNF because ProfessorName isn't a superkey

Solution:

- To **achieve BCNF**, we **decompose** Course table into **two separate tables**: Course table and Professor table (Updated tables)
- **Course table:**

CourseID	CourseTitle	Credits	ProfessorID	PrerequisiteID
C101	DBMS	4	P001	NULL
C102	OS	3	P002	C101

Normalization

Boyce-Codd Normal Form(BCNF)

- **Professor table:**

ProfessorID	Name
P001	Rao
P002	Sharma

- **Student, Department, Enrollment tables remains the same**
- **Student Table:**

StudentID	Name	Email	DOB	Major	DeptID
1001	Arjun	a@u.com	2002-05-01	CS	D1
1002	Meera	m@u.com	2001-08-11	IT	D2

Normalization

Boyce-Codd Normal Form(BCNF)

- **Department Table:**

DeptID	DeptName	Building
D1	CS	Block A
D2	IT	Block B

- **Enrollment Table:**

StudentID	CourseID	Grade
1001	C101	A
1001	C102	B+
1002	C101	B

Fourth Normal Form(4NF)



Normalization

Fourth Normal Form(4NF)

Fourth Normal Form(4NF)

- Fourth Normal Form (4NF), we need to **ensure** there are **no multi-valued dependencies**.
- Multi-valued dependencies occur when **one attribute** in a table **uniquely determines another attribute, independently of all other attributes**.
- In the current structure, there are **no obvious multi-valued dependencies** within a **single table**.
- Each **attribute appears to be correctly associated with the primary key** of its respective table.
- However, to illustrate the concept, let's consider the situation where **one CourseID in Course table has multiple Prerequisites**, this would **violate 4NF**.

Normalization

Fourth Normal Form(4NF)

- **Course Table Violating 4NF:**

CourseID	CourseTitle	Credits	ProfessorID	PrerequisiteID
C101	DBMS	4	P001	NULL
C102	OS	3	P002	C101
C201	Distributed Systems	3	P003	C101, C102

- Here, the **PrerequisiteID** column is a **multi-valued dependency** on **CourseID**.

Solution:

- **To achieve 4NF**, we should **separate PrerequisiteID** into a **new table**.
- The new table Prerequisite has attributes CourseID, PrerequisiteID

Normalization

Fourth Normal Form(4NF)

- Updated 4NF Tables:
- Course table:

CourseID	CourseTitle	Credits	ProfessorID
C101	DBMS	4	P001
C102	OS	3	P002
C201	Distributed Systems	3	P003

- Prerequisite table:

CourseID	PrerequisiteID
C101	NULL
C102	C101
C201	C101
C201	C102

Normalization

Fourth Normal Form(4NF)

- Professor, Student, Department, Enrollment tables remains the same

- Professor table:

ProfessorID	Name
P001	Rao
P002	Sharma
P003	Bharani

- Student Table:

StudentID	Name	Email	DOB	Major	DeptID
1001	Arjun	a@u.com	2002-05-01	CS	D1
1002	Meera	m@u.com	2001-08-11	IT	D2

Normalization

Fourth Normal Form(4NF)

- **Department Table:**

DeptID	DeptName	Building
D1	CS	Block A
D2	IT	Block B

- **Enrollment Table:**

StudentID	CourseID	Grade
1001	C101	A
1001	C102	B+
1002	C101	B

Fifth Normal Form(5NF)



Normalization

Fifth Normal Form(5NF)

Fifth Normal Form(5NF)

- **Fifth Normal Form (5NF)**, also known as **Project-Join Normal Form (PJNF)**, ensures that **every non-trivial join dependency** in the table is implied by the **candidate keys**.
- **5NF addresses cases** where the **table** may need to be **decomposed to remove join dependencies**.
- **These are dependencies** where a **table** can be **divided** into **smaller tables**, and **each smaller table** can be **joined back** to form the **original table without loss of information**.
- A **spurious tuple** in 5NF arises when **decomposed relations are joined back** and produce **invalid or unintended rows** not present in the original relation.
- It occurs due to **improper decomposition**, especially of complex relationships like ternary associations.

Normalization

Fifth Normal Form(5NF)

Scenario:

- Consider a scenario where you have a **ternary relation** like **Teaches(StudentID, CourseID, ProfessorID)** where a course can be taught by different professors to different students and a professor might teach different students in the same or different courses.

- Teaches Table:**

StudentID	CourseID	ProfessorID
S1	C1	P1
S1	C2	P2
S2	C1	P2

- Professor P1 teaches Course C1 to Student S1
- Professor P2 teaches Course C2 to Student S1 and Course C1 to Student S2

Normalization

Fifth Normal Form(5NF)

- Now, if we decompose this into **three binary relations**: **StudentCourse**(StudentID, CourseID), **CourseProfessor**(CourseID, ProfessorID), **StudentProfessor**(StudentID, ProfessorID)

- StudentCourse Table:**

StudentID	CourseID
S1	C1
S1	C2
S2	C1

- CourseProfessor Table:**

CourseID	ProfessorID
C1	P1
C2	P2
C1	P2

Normalization

Fifth Normal Form(5NF)

- **StudentProfessor Table:**

StudentID	ProfessorID
S1	P1
S1	P2
S2	P2

- If **StudentCourse** table and **CourseProfessor** table are **joined** using common column **CourseID**

StudentID	CourseID	ProfessorID	Remarks
S1	C1	P1	
S1	C1	P2	Spurious tuple
S1	C2	P2	
S2	C1	P1	Spurious tuple
S2	C1	P2	

Normalization

Fifth Normal Form(5NF)

- If **StudentCourse** table and **StudentProfessor** table are **joined** using common column **StudentID**

StudentID	CourseID	ProfessorID	Remarks
S1	C1	P1	
S1	C1	P2	Spurious tuple
S1	C2	P2	
S2	C1	P2	
S2	C2	P2	Spurious tuple

Normalization

Fifth Normal Form(5NF)

- If **CourseProfessor** table and **StudentProfessor** table are **joined** using common column **ProfessorID**

StudentID	CourseID	ProfessorID	Remarks
S1	C1	P1	
S1	C1	P2	Spurious tuple
S1	C2	P1	Spurious tuple
S1	C2	P2	
S2	C1	P2	

- Therefore, the ternary relationship Teaches(StudentID, CourseID, ProfessorID) **cannot be accurately reconstructed** from binary projections.
- That's why 5NF states that relations should only be decomposed if the join of the projections exactly equals the original relation.**

Sixth Normal Form(6NF)



Normalization

Sixth Normal Form(6NF)

Sixth Normal Form(6NF)

- 6NF deals with **decomposing a relation** into **irreducible components** such that **each component has only one non-trivial join dependency**.
- Every relation contains **only a single, non-trivial fact** that cannot be further decomposed without introducing temporal data.
- This form is used to **manage temporal data** and ensure that **every constraint is enforced by primary key constraints alone**.
- Used mainly when **tracking time-varying attributes**.
- To better understand 6NF, let's consider a scenario involving temporal data where we need to **track professors handling courses over time**.

Normalization

Sixth Normal Form(6NF)

Scenario: Consider the **Course table** which contains CourseID, CourseTitle, Credits, ProfessorID

- In this, the **course may be taught by different professors at different times**
- **Temporal attributes** such as ValidFrom, ValidTo are added for **tracking professors handling courses over time.** (ProfessorID changes over time)

CourseID	CourseTitle	Credits	ProfessorID	ValidFrom	ValidTo
C101	DBMS	4	P001	2024-01-01	2024-06-30
C101	DBMS	4	P002	2025-01-01	2025-06-30
C102	OS	3	P002	2024-01-01	2024-06-30
C102	OS	3	P003	2025-01-01	2025-06-30

- The above course table **repeats CourseTitle and Credits** even though they didn't change—violating normalization principles and increasing risk of inconsistency.

Normalization

Sixth Normal Form(6NF)

Solution:

- **Split the Course table into two:**
- One for course details that don't change over time.
- One for the professor assignment, with ValidFrom and ValidTo to track when each professor taught the course.
- **This makes the design cleaner, more flexible, and ready for time-based queries.**
- **Course Table:**

CourseID	CourseTitle	Credits
C101	DBMS	4
C102	OS	3

Normalization

Sixth Normal Form(6NF)

- **CourseProfessor Table:**

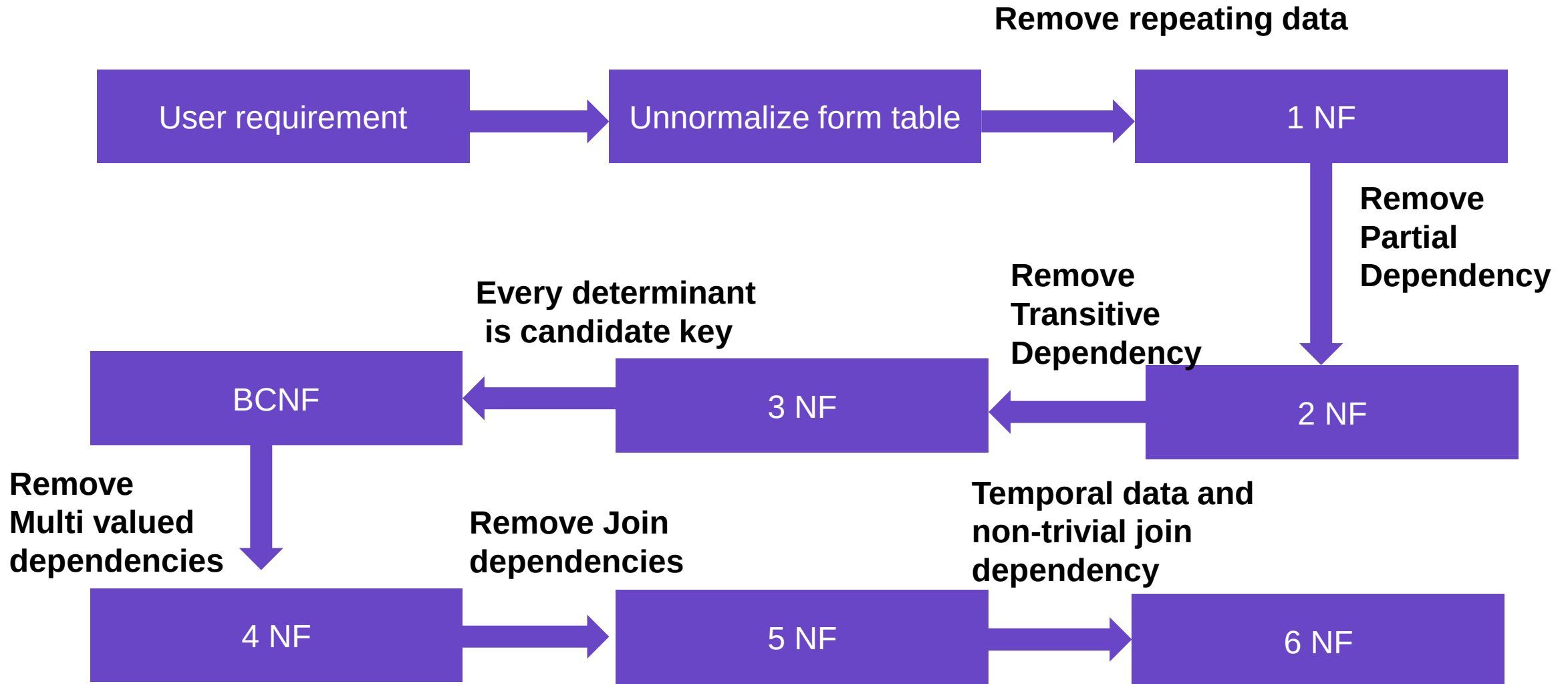
CourseID	ProfessorID	ValidFrom	ValidTo
C101	P001	2024-01-01	2024-06-30
C101	P002	2025-01-01	2025-06-30
C102	P002	2024-01-01	2024-06-30
C102	P003	2025-01-01	2025-06-30

Why This Is 6NF?

- **Each table holds exactly one fact:** Course table holds static data about the course, CourseProfessor table holds the professor assignment, which can change over time.
- **No redundancy:** If only the professor changes, we don't repeat CourseTitle or Credits.
- **Supports temporal queries:** You can now query: "Who taught Data Structures in 2021?"

Normalization

Normalization Forms: Summary



Denormalization in databases



Denormalization in databases

Introduction

- Process of intentionally **introducing redundancy** into a database design **to improve read performance**.
- Typically done **after normalization** (when the data model is already in at least 3NF).
- Often used in data warehouses, reporting, or read-heavy applications.
- **Trade-off:** Storage space and redundancy vs. faster read queries.

Four typical techniques:

1. **Adding redundant columns** - Avoid joins by copying commonly accessed attributes
2. **Storing Derived or Aggregated Data** - Avoid recomputing values on the fly
3. **Precomputed Summary Tables** - Store query results in a separate table to speed up complex reports
4. **Duplicating Data for Faster Access** - Reduce joins for frequent lookups or UI rendering

Denormalization in databases

Denormalization : Example

Consider Normalized Schema in University database

- Student(StudentID, Name, Email, DOB, Major, DeptID)
- Course(CourseID, CourseTitle, Credits, ProfessorID)
- Prerequisite(CourseID, PrerequisiteID)
- Professor(ProfessorID, Name)
- Department(DeptID, DeptName, Building)
- Enrollment(StudentID, CourseID, Grade)

Denormalization in databases

Denormalization : Example

1. Adding redundant columns

- **Goal:** Avoid joins by copying commonly accessed attributes.

Example:

- **Add DeptName to Student table** so that when displaying a student's department name, **no need to join with Department table.**
- Therefore, Student(StudentID, Name, Email, DOB, Major, DeptID, DeptName).
- Useful for reporting dashboards and forms.
- **Trade-off: Risk of inconsistency** - if DeptName changes in Department, you must update all Student rows with the old name.

Denormalization in databases

Denormalization : Example

2. Storing Derived or Aggregated Data

- **Goal:** Avoid recomputing values on the fly.

Example:

- When displaying student's details, every time we have to compute total credits earned by the student.
- So, **Add TotalCreditsEarned to the Student table** to avoid recomputing values.
- Therefore, Student(StudentID, Name, Email, DOB, Major, DeptID, TotalCreditsEarned).
- **TotalCreditsEarned** attribute is derived from summing credits of all enrolled and passed courses.
- It saves expensive joins and aggregation during each query.
- **Trade-off:** Needs update logic every time student completes a course.

Denormalization in databases

Denormalization : Example

3. Precomputed Summary Tables

- **Goal:** Store query results in a separate table to speed up complex reports.

Example:

- Create **DepartmentStats** table with attributes DeptID, TotalStudents, TotalCourses, AvgGrade.
- It helps to generate admin reports quickly.
- It avoids running expensive joins and groupings over live data.
- **Trade-off:** Requires ETL/trigger to keep this table in sync with live data.

Denormalization in databases

Denormalization : Example

4. Duplicating Data for Faster Access

- **Goal:** Reduce joins for frequent lookups or UI rendering.

Example:

- **Store CourseTitle and ProfessorName inside Enrollment** to avoid joining with Course and Professor tables when displaying enrollment history.
- Therefore, Enrollment(StudentID, CourseID, Grade, CourseTitle, ProfessorName)
- **Trade-off:** Any title or name change must be cascaded manually.

Denormalization in databases

Denormalization : Example

Disadvantages

- Increased storage space.
- Data anomalies (inconsistencies if not carefully managed).
- Complexity in data maintenance (e.g., updates, inserts).
- May violate data integrity.

Use denormalization only when:

- **Performance demands it** (e.g., frequent reports or dashboards).
- You're using a **read-optimized system** (OLAP).
- Data is **rarely updated**, or **updates are well-controlled**.

THANK YOU